

Infrastructure Resilience Digital Twin

A Neuro-Symbolic Cascading Failure Simulator for Indian Infrastructure Networks

Built on the VARUNA Eco-Hydro-AI Platform · PES University · MIT Tech Tattva

Hackathon Problem Statement 11 — Sustainable Cities and Communities: Cascading Failure

0. Document Purpose

This document consolidates the full architecture, technical rationale, and scoping decisions for the system, across a full design process that started from the original VARUNA drainage-design platform and evolved it into a dedicated infrastructure cascading-failure simulator. It is intended as the single reference for the team: what each component is, why it exists, what it depends on, and what it honestly cannot yet claim.

1. The Problem, Restated Precisely

The problem statement asks for a system that:

1. Represents a selected infrastructure network (roads, bridges, hospitals, utilities) as an interconnected system, not isolated assets.
2. Explores how disruptions propagate through that network.
3. Lets users introduce one or more failures.
4. Estimates the wider effects of those failures.
5. Identifies disproportionately important assets or connections.
6. Compares alternative scenarios.

The core design decision this entire architecture is built around: VARUNA, the original flood-prediction platform, solves a *different* problem — physics-based hydrological simulation on a single-domain terrain graph. It does not, on its own, satisfy any of the six requirements above. Rather than discard it, VARUNA is repurposed as a **hazard generator** feeding into a new, purpose-built **cascading infrastructure failure engine**, which is what this document describes.

2. Governing Design Philosophy

Three principles drive every architectural decision below.

2.1 Topology mismatch is the root problem, not a detail. Fluid dynamics propagates through *spatial* adjacency — water only interacts with immediately neighboring terrain. Infrastructure cascades propagate through *functional* dependency — a substation failure can knock out a hospital 8km away with nothing failing in between. A raster/grid graph with 8-neighbor (D8) adjacency structurally cannot represent this. The fix is not a

parameter change; it is a change of graph type, from a spatially-local raster graph to a **heterogeneous, vector-based dependency graph**.

2.2 Symbolic and neural components play different, complementary roles — not sequential stages. The deterministic graph-algorithm engine (Tier 0/1) is the system's ground truth generator and final verifier. The neural surrogate (Tier 2) exists purely to make exploring that ground truth fast enough for live interaction and large-scale Monte Carlo sampling. The neural layer is never permitted to be the final, unchecked authority on any output.

2.3 Verification is not validation, and the system must say so. Every check in this architecture — conservation laws, capacity bounds, confidence thresholds — confirms *internal consistency*: that the system's outputs obey the rules it was given. None of these checks confirm those rules describe *reality*, because no historical dataset of real cascading infrastructure failures exists for rural Indian villages. This distinction is treated as a first-class, explicitly stated property of the system, not a hidden limitation. See Section 10.

2.4 Hazard causation and cascade propagation are strictly separated. What causes a node to fail (flood, fire, landslide, a sinkhole, a construction accident, a civil disruption) is fundamentally different physics per hazard type. What happens once a node has failed is identical regardless of cause. The architecture enforces this as an interface: every hazard type implements a small, swappable module that outputs one standardized signal per affected node/edge — a binary failed flag, or a $P(\text{failure})$ value, optionally evolving across discrete rounds for a spreading hazard (flood, fire) or given once for an instantaneous point-event hazard (sinkhole, collapse, closure). Stages 2, 3, 6, and 7 never need to know which hazard module produced that signal. This is what allows the system to generalize beyond VARUNA's flood case without re-architecting anything downstream — see Section 5, Stage 4/5, and the table above.

3. Hazard Module Interface (generalizing beyond flood)

VARUNA's DUALFloodGNN is the *first instance* of a hazard module under this interface, not the only one the architecture supports. Any new hazard type only needs to implement the contract below; nothing else in the pipeline changes.

HAZARD MODULE CONTRACT

Input: whatever data that specific hazard needs

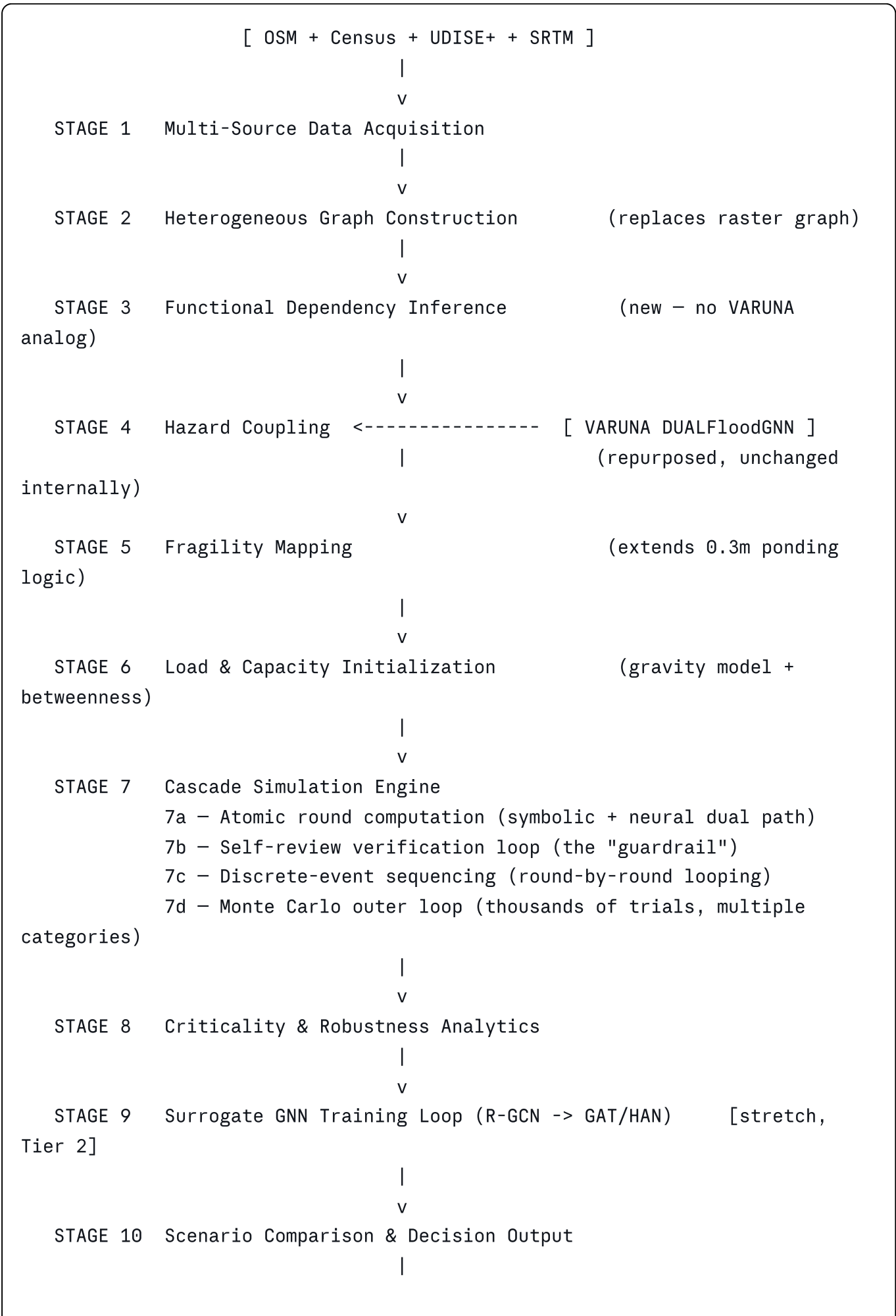
Output: per node/edge, either

- a binary failed flag, or
- $P(\text{failure})$, optionally evolving over discrete rounds (spreading hazards: flood, fire) or given once (point-event hazards: sinkhole, collapse, closure)

Hazard type	Hazard-specific input	How it produces the P(failure) signal
Flood (built)	SRTM/LiDAR DEM, rainfall scenario	DUALFloodGNN depth prediction -> Stage 5 sigmoid fragility curve
Fire	Building density, wind, ignition point	Spread-radius or cellular-automaton model -> nodes inside the advancing perimeter at round t
Road destruction (sinkhole, collapse, closure, accident)	A specific node/edge identifier	Direct, instantaneous binary failure — no physics model needed; identical mechanism to the manual-failure entry point in the user flow
Landslide	Slope, TWI, rainfall, soil saturation	Reuses VARUNA's existing terrain-processing pipeline (slope/TWI already computed for flood) with a landslide-specific fragility threshold — relevant for hill-region deployments (e.g. Western Ghats), not flat-terrain cities

Payoff: adding a new hazard type is bounded, additive work — one new module implementing the contract above — not a re-architecture of the graph, dependency inference, load initialization, or cascade engine. This also enables compound multi-hazard Monte Carlo trials (Stage 7d) — e.g. a flood-driven road failure and an unrelated simultaneous fire — by simply running two hazard modules into the same trial, since both produce the same standardized signal the cascade engine already consumes.

4. High-Level Architecture



5. Stage-by-Stage Architecture

STAGE 1 — Multi-Source Data Acquisition

Purpose: Assemble real infrastructure and population data for a selected district, replacing VARUNA's single-source DEM input with multiple sources appropriate to a network problem.

Data sources:

Source	Provides	Notes
OpenStreetMap (osmnx)	Road topology, bridges, hospitals, schools, substations, water infrastructure	Free; rural tagging completeness varies
Census 2011 Village Directory / SECC	Habitation locations, population counts	Fallback: WorldPop 100m gridded estimate
UDISE+	School directory	Cross-validates OSM school tags
data.gov.in health facility listings	Hospital/clinic directory	Cross-validates OSM hospital tags
SRTM 30m DEM	Terrain for VARUNA's hazard layer	Same source as original VARUNA
IRC / PMGSY road standards	Road capacity classification	Grounds Stage 6 load calibration in engineering standards, not inferred traffic data

Output artifacts:

```
data/raw/{district}_osm_network.graphml
data/raw/{district}_facilities.geojson
data/raw/{district}_population.geojson
```

Known gap: OSM/UDISE+ coverage for rural India is incomplete — minor roads, hand pumps, and small substations may be missing entirely. This is stated as a limitation, not silently absorbed.

STAGE 2 — Heterogeneous Graph Construction

Purpose: Replace VARUNA's single-node-type raster graph with a typed, multi-relational graph that can represent non-adjacent functional dependencies.

Technical container: PyTorch Geometric's `HeteroData` object — a direct generalization of the `Data` object VARUNA already used, storing a separate feature matrix per node type and a separate edge index per relation type.

Node types and features:

```
road_segment: {length_m, highway_class, pmgsy_capacity_proxy}
bridge:       {length_m, flood_fragility_depth_threshold}
hospital:     {capacity_beds, has_backup_power}
school:       {student_count}
substation:   {load_capacity_mw}
habitation:   {population}
```

Edge types (relations):

```
(road_segment, connects_to, road_segment) -- physical/spatial
(bridge, carries, road_segment)           -- physical/spatial
(hospital, powered_by, substation)         -- functional dependency
(water_pump, powered_by, substation)       -- functional dependency
(habitation, served_by, hospital)          -- functional dependency
(habitation, accesses, road_segment)       -- functional dependency
```

Why vector math replaces D8 adjacency: Every node keeps a real `pos` attribute (latitude/longitude). Edge length and bearing are computed via geodesic (haversine) distance between actual coordinates, not fixed grid-cell steps. This is what allows a dependency edge to connect two nodes that are nowhere near each other spatially — the entire capability D8 adjacency structurally lacked.

Input: Stage 1's raw data files. **Output:** `data/processed/{district}_hetero_graph.pt`

Known gap: Facility-to-network snapping (assigning a hospital to its nearest road node) is a nearest-neighbor heuristic, not a verified mapping.

STAGE 3 — Functional Dependency Inference

Purpose: No dataset directly states "this hospital's power comes from that substation." This stage infers it with explicit, stated rules — the same discipline VARUNA used for its Manning's-roughness lookup table.

The scale problem, and why it doesn't actually occur: Naively checking every node against every other node for a possible dependency is an $O(n^2)$ comparison —

intractable even at a few thousand nodes. This is avoided entirely, not filtered down after the fact, by exploiting that real infrastructure dependency is inherently local and typed. The inference process runs in two stages per node, never touching the full pairwise space:

STAGE A - Candidate generation (cheap, spatial-index-based)

Build a KD-tree / BallTree (haversine metric) per facility type.
For each node needing a dependency, query its K nearest candidates of the relevant type by straight-line distance -- $O(\log n)$ per query.

STAGE B - Rule application (only on the small candidate set from A)

Refine the K candidates by real road-NETWORK distance (Dijkstra), not straight-line distance, since straight-line-nearest can be wrong (e.g. across a river with no bridge).

Apply the type-specific rule (see table below) to pick the final assignment(s) from the refined candidate set.

python

```
for each hospital h:
    candidates = spatial_index.query(substations, k=3, metric="haversine")
    for c in candidates:
        c.network_distance = dijkstra(h.nearest_road_node, c.nearest_road_node)
    assigned_substation = min(candidates, key=lambda c: c.network_distance)
```

Rules by dependency type:

Dependency edge	Rule	Cardinality
<u>hospital/school/pump -></u> <u>powered_by -></u> <u>substation</u>	Nearest by network distance among K=3 spatial candidates; capacity-aware fallback (skip to next-nearest if the closest substation's assigned load already exceeds its stated capacity)	1:1
<u>habitation -> served_by</u> <u>-> hospital</u>	Nearest by network distance, OR Voronoi/Thiessen catchment assignment (partitions the whole district into non-overlapping facility catchment areas — more principled than an arbitrary distance cutoff)	1:many (top-2, so a single hospital's failure doesn't automatically isolate every dependent village)
<u>habitation -> served_by</u> <u>-> school</u>	Same as above	1:many (top-2)
<u>habitation/facility -></u> <u>accesses -></u> <u>road_segment</u>	Nearest road node by straight-line distance (a geometric snap, not a network-distance question)	1:1

Threshold parameters (stated explicitly, not hardcoded silently): K (number of spatial candidates considered, typically 3), a maximum network-distance cutoff beyond which no dependency is assigned (e.g. 15km — treated as genuinely unserved rather than forced into a bad assignment), and the substation capacity-headroom threshold for the power-assignment fallback.

Validation pass, run after Stage 3 completes: Flag any assignment where network distance exceeds ~3x straight-line distance (signals a data/routing gap, e.g. a missing OSM road segment), and flag any node with zero candidates within the distance cutoff (a genuinely unserved area worth surfacing, not silently dropping).

Scale check, for a realistic district (50 habitations, 6 hospitals, 14 schools, 3 substations, a few hundred road segments): roughly 300 dependency edges total, not millions — because K-nearest assignment scales linearly with node count, unlike all-pairs comparison.

Input: The graph from Stage 2. **Output:** The same graph, now with dependency edges populated.

Known gap: These are rule-based assumptions, not measured facts. Stated explicitly in every output that depends on them, mirroring VARUNA's own honesty about its is_protected ecological flag. Two specific, named failure modes: boundary effects (a habitation near the district edge may have its true nearest facility physically outside the mapped area) and silent bad assignments at OSM data gaps (a missing road segment can

force a network-distance calculation onto a real but unmapped-shorter path, producing an assignment that looks defensible but is wrong on the ground).

STAGE 4 — Hazard Coupling (VARUNA Integration)

Purpose: This is where the original VARUNA system re-enters — unchanged internally, repurposed as an upstream hazard generator rather than the system's core.

How it works: VARUNA's `DUALFloodGNN` (Physics-Informed GNN embedding the Shallow Water Equations via automatic differentiation, per the original VARUNA report) produces per-node flood depth predictions `h(x,y)` exactly as before. A GIS spatial join (via `rasterio`/`geopandas`) overlays this depth raster onto the road/bridge line geometry from Stage 2, producing a maximum predicted depth per road/bridge segment.

Input: SRTM DEM (or drone LiDAR, per VARUNA's original tiered ground-filtering pipeline); the road/bridge geometry from Stage 2. **Output:** Per-segment maximum flood depth, joined onto the infrastructure graph.

Known gap (inherited from VARUNA): Depth predictions are physically consistent (satisfy mass conservation and gravity alignment) but not calibrated against measured flood data — VARUNA's own stated Bottleneck 1. This uncertainty propagates into every hazard-driven failure scenario downstream.

STAGE 5 — Fragility Mapping

Purpose: Convert hazard values (or any stress condition) into asset failure probabilities — the direct structural extension of VARUNA's `depth > 0.3m` ponding threshold, generalized across asset types and expressed as a smooth curve rather than a hard cutoff.

Why a curve, not a cutoff: A hard threshold (road fails exactly at 0.3m, survives at 0.29m) is not physically realistic and gives no way to express "probability of collapse," which the system needs for its confidence-band UI. A **fragility curve** (standard in structural/seismic/reliability engineering) is used instead:

```
utilization = current_load / capacity
```

```
P(failure) = sigmoid( k * (utilization - 1) )
```

```
utilization << 1 -> P(failure) near 0
```

```
utilization == 1 -> P(failure) = 0.5    (the nominal threshold)
```

```
utilization >> 1 -> P(failure) near 1
```

```
k controls how sharp or gradual the transition is
```

Thresholds used, by asset type (grounded in named engineering standards, not invented):

road (light vehicle):	depth > 0.15m	-> impassable
road (heavy vehicle):	depth > 0.30m	-> impassable
bridge approach:	depth > 0.50m sustained >6h	-> failed
substation (flood):	within flood extent	-> failed

Input: Stage 4's per-segment depth values; Stage 2's asset-type attributes. **Output:** A $P(\text{failure})$ value per asset, per hazard scenario.

Known gap: Thresholds for roads follow documented vehicular-passability standards; some asset classes (e.g., substation flood tolerance) rest on simplifying assumptions that are stated but not independently measured for this specific domain.

STAGE 6 — Load & Capacity Initialization

Purpose: Give every road/edge in the network a starting number for "how much is normally flowing through here" (load) and "how much can it handle before breaking" (capacity) — without hand-assigning arbitrary numbers, and in a way that stays dynamic as the network changes.

Why this needed its own stage: Early drafts of this architecture considered borrowing an unrelated equation (a combat-attrition model) to generate this number. That approach was rejected after review — no established connection exists between military attrition modeling and infrastructure load, and it would not withstand technical questioning. The mechanism below uses only established, domain-appropriate techniques.

Step-by-step mechanism:

1. **Trip generation.** Every habitation and facility gets a "trips produced" / "trips attracted" number, derived directly from Stage 1 population and facility-capacity data (a village of 1,200 people generates roughly proportional trip demand; a 50-bed hospital attracts roughly proportional visits).
2. **Trip distribution — the gravity model.** A standard transportation-planning technique (the second step of the classic four-step travel demand model) estimates trips between every pair of places:

$$T_{ij} = k * (O_i * D_j) / f(c_{ij})$$

O_i = trips produced at node i (from population)

D_j = trips attracted to node j (from facility capacity)

c_{ij} = travel cost (distance/time) between i and j

$f(c_{ij})$ = distance-decay function (farther apart = fewer trips)

3. **Traffic assignment.** Each estimated origin-destination trip count is routed onto the real road network via shortest-path assignment (Dijkstra), and trip counts are

summed along every road segment they pass through. This produces the **initial load**, $L_i(0)$, per segment — derived from real demand data, not guessed.

4. **Capacity, from the Motter-Lai formulation.** Following the standard cascading-failure literature (Motter & Lai, 2002), capacity is set as:

$$C_i = (1 + \alpha) * L_i(0)$$

where α is a tolerance parameter (a segment's breaking point sits somewhat above its normal load, reflecting that infrastructure is built with some headroom).

5. **Dynamic recomputation, every round.** After Stage 6 runs once to set the fixed baseline $L_i(0)$ and C_i , the *current* load $L_i(t)$ at every later round is recomputed as **betweenness centrality** on the surviving graph topology — the count of shortest paths currently passing through node i . This is what keeps load dynamic without needing to re-run the gravity model every round: as topology changes (things fail, routes change), betweenness automatically redistributes.

Input: Population and facility data (Stage 1); the graph (Stage 2/3). **Output:** $L_i(0)$ and C_i per node/edge, refreshed as $L_i(t)$ every cascade round.

Worked example:

Village 1 (pop 1200) -> Hospital A: gravity model estimates 340 trips/day
Village 2 (pop 800) -> Hospital A: gravity model estimates 210 trips/day
Both routes use Road 14 -> Road 14 starting load $L(0) = 550$ trips/day
Road 14 capacity $C = 550 * 1.2 = 660$ trips/day

Known gap: The α tolerance buffer and the gravity model's distance-decay function are reasonable, literature-grounded estimates, not measurements calibrated to this specific district's real traffic behavior.

STAGE 7 — Cascade Simulation Engine

This is the system's computational core. It has four sub-parts, described in the order they execute.

7a — Atomic Round Computation (dual path)

Every round of the cascade is computed two ways in parallel:

Symbolic path (Tier 0/1 — NetworkX, Motter-Lai model):

1. Identify nodes/edges that failed in the previous round
2. Find their carried load
3. Redistribute that load onto surviving shortest paths (Dijkstra)
4. Recompute betweenness-based load on the new topology
5. Compare new load against fixed capacity (Stage 6)
6. Any node/edge exceeding capacity -> newly failed

Exact and deterministic; the system's ground truth. Computationally heavier — real graph-algorithm work every round.

Neural path (Tier 2 — GAT surrogate, PyTorch Geometric):

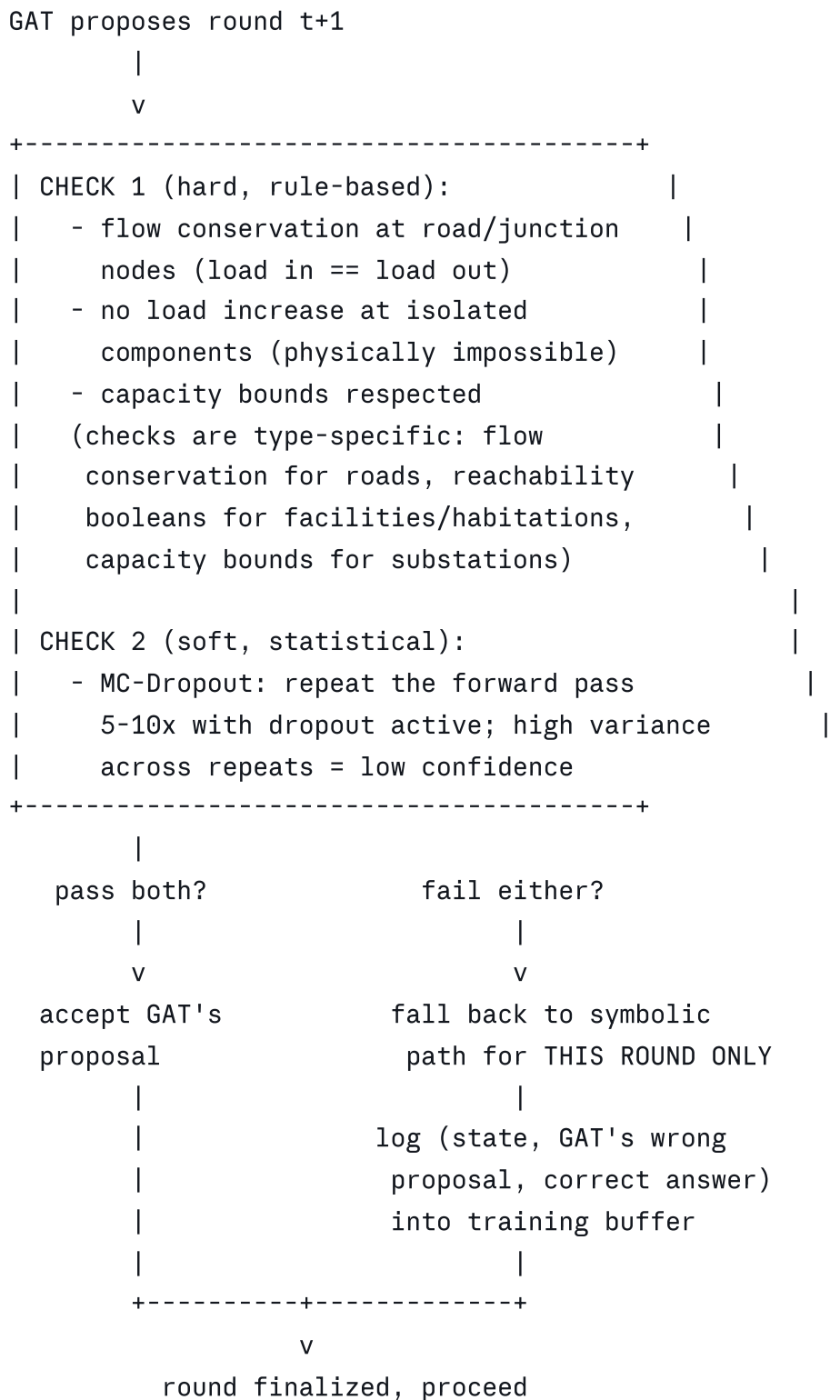
1. Encode current graph state (features + failed mask + current load)
2. GAT forward pass: per-edge attention weights, computed from both endpoints' current features, determine how much each neighbor's state influences a node's prediction
3. Output head: predicted failure probability per node

Approximate; fast (a single matrix-multiplication pass regardless of graph size). Trained entirely on the symbolic path's own historical outputs — it can only ever approximate Tier 0/1, never exceed it in correctness.

Why a GAT specifically, and what attention buys: Plain relation-typed message passing (R-GCN) gives every dependency of the same type equal weight — every powered_by edge treated identically. Attention lets the model learn that some specific dependencies matter more than others (a hospital with no backup power should weight its power dependency more heavily than one with backup) — and, importantly, the resulting attention weights are directly inspectable after training, giving a free, learned "which connections matter most" signal for Stage 8.

7b — Self-Review Verification Loop (the guardrail)

Every GAT proposal is checked before being trusted:



Why this catches some errors and not others: Conservation checks catch outputs that are internally impossible (hallucination). They cannot catch outputs that are internally consistent but simply wrong because they faithfully reproduce a flawed underlying assumption (e.g., a mis-calibrated capacity number). This distinction is treated as an explicit, named limitation, not glossed over.

Why fallback is per-round, not per-trial: Re-running the entire trajectory symbolically every time GAT falters anywhere would erase most of the speed advantage. Falling back

only for the specific failed round preserves amortized speed while guaranteeing every accepted state passed a real check.

7c — Discrete-Event Sequencing

```
round 0:  apply seed failure(s)
round 1:  7a + 7b compute new state from round 0
round 2:  7a + 7b compute new state from round 1
...
round t:  check — any NEW failures this round?
          NO  -> converged; trial ends
          YES -> continue
(hard cap at ~15-20 rounds as an oscillation safeguard)
```

Why round-by-round, not a single before/after diff: Three concrete reasons — it enables the animated dashboard reveal; it captures order-dependence (which asset fails third versus fifth genuinely changes downstream outcomes, since later rounds redistribute load across whatever survived earlier rounds); and it exposes intermediate stress levels at nodes that never actually failed, a richer signal for Stage 8's criticality ranking than a static snapshot provides.

7d — Monte Carlo Outer Loop

Trial category	Seed generation	Question
answered		
-----	-----	-----

Random single-point fragility	uniform random node/edge failure	baseline
Random multi-point event risk	random k-node simultaneous failure	compound-
Hazard-driven	VARUNA flood depth -> Stage 5	"what
does this monsoon	fragility threshold crossing	
scenario actually break"		
Targeted/worst-case	fail highest-centrality node first	
adversarial/worst-case bound		
Parameter-perturbed	same seed, capacity scaled 80%-120%	
sensitivity band (honest		
uncertainty communication)		

Each category runs as its own batch of thousands of independent trials, each executing the full 7a→7b→7c loop from its own seed to its own convergence. The amortized speed of the neural path is what makes this trial volume achievable within hackathon compute/time

budgets — most rounds resolve via a millisecond forward pass, with only the uncertain minority falling back to the slower exact engine.

STAGE 8 — Criticality & Robustness Analytics

Purpose: Turn thousands of completed trials into decision-grade numbers, and directly answer "identify disproportionately important assets or connections."

Metrics computed:

- **Empirical criticality ranking:** how often each asset appears in worst-outcome trials across the Monte Carlo batch — stronger than static centrality because it's frequency-weighted across simulated scenarios, not a single topological snapshot.
- **Static betweenness centrality / articulation points** (`networkx`): structural importance independent of any specific failure.
- **Robustness curve:** size of the largest connected component as assets are removed in criticality order versus random order — a classic, cheap, visually compelling way to show targeted-failure fragility versus random-failure robustness.
- **Algebraic connectivity (Fiedler value):** a single scalar resilience score per scenario, via graph Laplacian eigenvalues (`scipy`/`networkx`), enabling a one-number "resilience dropped from X to Y" statement.
- **GAT attention overlay (if Stage 9 is built):** learned per-edge importance, rendered directly on the map.

Input: All completed Monte Carlo trajectories from Stage 7d. **Output:** A ranked criticality list, a robustness curve, a Fiedler-value distribution.

Known gap: These rankings are statistically stable summaries of the *model's own behavior*, not validated statements about the real world — see Section 10.

STAGE 9 — Surrogate GNN Training Loop (*stretch goal, Tier 2*)

Purpose: Make the neural path in Stage 7a progressively more accurate and reduce the fallback rate over time.

PHASE 1 (offline): Run the symbolic engine (7a, Path A) for many Monte Carlo trials in full, every round, no shortcuts. Slice every consecutive (state_t, state_{t+1}) pair into a training example.

PHASE 2 (train): Train the GAT with:

- supervised loss against the symbolic ground truth
 - flow-conservation auxiliary loss (load in $\approx$ load out)
 - capacity-violation auxiliary loss (penalize predicted load > capacity)
- (these are the direct structural siblings of VARUNA's own continuity and gravity-alignment loss terms)

PHASE 3 (deploy): Use the trained GAT for the bulk of new trials; every 7b fallback event is logged and periodically used to retrain the model — an active-learning loop where the model improves precisely where it is currently weakest.

Architecture sequencing: Build a simpler R-GCN (fixed weight per relation type) first and validate it works end-to-end before attempting GAT's per-edge attention — the correctness risk of getting heterogeneous attention right under time pressure is real and should not be taken on simultaneously with validating the base surrogate concept.

Known gap: The surrogate's ceiling is exactly the symbolic engine's own assumptions — it cannot become more correct than what it was trained to imitate, only faster.

STAGE 10 — Scenario Comparison & Decision Output

Purpose: Directly answer "compare alternative scenarios."

How it works: Run baseline (no failure), then named scenarios (a specific failure; a failure plus a modeled repair/mitigation — including a VARUNA-designed drain, represented as an edited fragility attribute on a road node), diffing Stage 8's metrics across each.

Output example:

	Baseline	Bridge 7 fails	+ repaired in 6h
Pop. isolated	0	4,120	850
Facilities cut off	0	3	1
Robustness (Fiedler)	0.68	0.41	0.59

Known gap: Every additional scenario is a full re-run of the Monte Carlo batch, not a cheap diff — comparing many scenarios has a real compute cost.

6. User Flow, Mapped to Architecture

User-facing step	Stage(s)	What the user sees
1. Select district	1, 2, 3	Graph assembled from a typed district name
2. Examine relationships	(display only, reads Stage 2/3 output)	Clickable network map
3. Introduce a failure	4, 5, 6	Manual click or hazard-driven (rainfall) seed
4. Watch propagation	7 (a-c)	Round-by-round animated cascade
5. Estimate wider effects	(single-trial analytics on Stage 7 output)	Population isolated, facilities cut off, connectivity score
6. Identify critical assets	7d, 8	Ranked criticality list, robustness curve
7. Compare scenarios	10	Side-by-side scenario table

7. Technology Stack

Component	Tool / Library	Role
Road/facility data	<code>osmnx</code> (OpenStreetMap)	Real network topology
Population data	Census 2011 / SECC / WorldPop	Demand estimation input
Terrain / hazard	SRTM 30m DEM, VARUNA <code>DUALFloodGNN</code>	Flood depth prediction
Graph container	PyTorch Geometric <code>HeteroData</code>	Typed multi-relational graph
Symbolic cascade engine	<code>networkx</code> (shortest_path, betweenness_centrality, connected_components, articulation_points)	Ground truth cascade computation
Neural surrogate	PyTorch Geometric <code>RGCNConv</code> -> <code>GATConv</code>	Fast amortized cascade prediction
Uncertainty estimation	MC-Dropout	Confidence-based fallback trigger
Resilience metrics	<code>scipy</code> / <code>networkx</code> (Laplacian eigenvalues)	Fiedler value / algebraic connectivity
Dashboard	Streamlit + a mapping layer (deck.gl or equivalent)	Interactive user-facing interface
GIS coupling	<code>rasterio</code> , <code>geopandas</code>	Joining VARUNA's raster hazard output onto vector infrastructure geometry

8. Bottlenecks and Optimal Solutions

#	Bottleneck	Why it matters	Optimal solution adopted
1	Raster graph cannot represent non-local dependency	Root cause of the entire VARUNA-to-cascade pivot	Heterogeneous graph (Stage 2) with vector-computed edges, replacing D8 adjacency entirely
2	No cross-sector, rural-scale cascading-failure dataset exists — though the core <i>mechanism</i> (Motter-Lai overload redistribution) is empirically grounded in the power-grid and urban-road domains, including India's own July 2012 Northern/Eastern/North-Eastern grid collapse (~620-670M affected, official Enquiry Committee report)	Cannot validate the project's specific novel extension (cross-sector, rural-scale) against reality, even though the underlying mechanism is not unvalidated in general	Precise verified / domain-adjacently-validated / not-yet-validated-for-this-extension framing (Section 10); run the prototype benchmark strategy against real data before applying it to the rural target (Section 11); calibrate inputs (IRC/PMGSY) for this project's specific domain rather than claim the cross-sector extension itself is validated
3	Neural surrogate can hallucinate impossible states	Risk of confidently wrong output reaching the user	Layer 7b self-review loop: hard conservation checks + soft confidence checks, symbolic fallback on failure
4	Conservation checks catch gross errors, not subtle wrong-but-consistent ones	The more dangerous failure mode is the one that passes checks	Named explicitly as a residual limitation; not solvable with current architecture, stated honestly rather than hidden
5	Symbolic engine is slow to re-run at scale for thousands of Monte Carlo trials	Limits statistically meaningful trial counts within time budget	Neural surrogate (Stage 7a Path B) for amortized inference; fallback only for uncertain rounds, not full re-runs
6	Static, hand-assigned load values are not defensible	"Why is this road's capacity this number?" is a weak answer under questioning	Gravity-model trip distribution + traffic assignment (Stage 6), grounded in real population/facility data, not guessed
7	Hard failure thresholds are physically unrealistic and give no uncertainty signal	Cannot express "probability of	Sigmoid fragility curves (Stage 5) instead of binary cutoffs

#	Bottleneck	Why it matters	Optimal solution adopted
		collapse," a stated system goal	
8	Rural India lacks telemetry/traffic data sources common in Western case studies (e.g., Uber Movement, which is discontinued; TomTom probe data, sparse in low-connectivity rural areas)	Naive calibration plans fail for this specific geography	Use IRC/PMGSY engineering design standards instead — a more defensible, domain-appropriate source than sparse or unavailable telemetry
9	Borrowed equations from unrelated domains (e.g., combat-attrition models) lack a justified physical mapping to infrastructure load	Fragile under technical questioning; formal resemblance is not mechanistic justification	Rejected in favor of established, domain-correct techniques: betweenness centrality (network science) and gravity-model trip distribution (transportation engineering)
10	OSM/UDISE+ coverage is incomplete for rural India	Parts of the real network are invisible to the system	Stated explicitly as a data-availability limitation, not silently absorbed; cross-validate facility data against multiple sources where possible
11	Functional dependency edges (e.g., hospital-substation links) are inferred, not measured	Downstream cascade results inherit unverified assumptions	Nearest-neighbor inference rules stated explicitly (Stage 3); UI should visually distinguish measured (OSM) edges from inferred (rule-based) edges
12	D8-style graph pathfinding (in the original VARUNA context) produces "staircase" alignments	Not directly relevant to the cascade engine, but retained as a known VARUNA-side limitation if its drain-design output is reused as a mitigation scenario	Acceptable for planning-level output; noted as a limitation if used for construction-ready drawings

9. Scoping by Timeline

36-Hour Hackathon Build

- Stages 1-3, 5, 6, 7a (**symbolic path only** — Path A), 7c, 7d, 8, 10.
- **Skip:** the neural surrogate (Stage 7a Path B), Stage 9 entirely, and VARUNA hazard coupling (Stage 4) if time is tight — state these as the 22-day roadmap rather than attempting a rushed, unvalidated version.
- Team split (4 people): data/graph construction; fragility + symbolic cascade engine; criticality/robustness analytics; dashboard + scenario comparison + pitch narrative.

22-Day Build

- Add Stage 4 (VARUNA hazard coupling) once Stages 1-3 and 5-8 are validated.
- Add Stage 9 (surrogate GNN) — R-GCN baseline first, GAT/attention upgrade only if the baseline is solid with days to spare.
- Add the self-review verification loop (7b) once both the symbolic engine and a working surrogate exist independently — do not build the fallback logic and the base surrogate simultaneously.
- Reserve final days for validation, sensitivity-analysis-driven confidence bands (Method C style), and demo rehearsal.

10. Honest Limitations — Stated for the Pitch, Not Discovered by Judges

This system is **verified, and partially, domain-adjacently validated — but not validated for its specific novel contribution**. This is a more precise and more defensible claim than "no data exists," and the precision is worth stating explicitly rather than glossed over.

What is genuinely validated: the core cascading mechanism this architecture is built on — threshold-based overload redistribution (the Motter-Lai model) — is not a novel or unvalidated invention. It has real-world empirical grounding in the power-grid domain, checked against real historical blackout data (including published work validating cascading-failure models against real blackout-size distributions from real grids). It is also grounded in real, extensively documented cascading events, including the **July 2012 India Northern/Eastern/North-Eastern grid collapse** — the largest blackout in world history by population affected (~620–670 million people across 20+ states), triggered by exactly the overload-redistribution "spiral breakdown" sequence this model formalizes, and investigated in an official Government of India Enquiry Committee report.

What remains genuinely unvalidated — and is this project's actual novel contribution: that validation exists for a **single-domain, homogeneous network** (electrical transmission only) at **national/regional grid scale**, largely at the level of **aggregate statistical properties** (blackout-size distribution shape) rather than granular, node-by-node trajectory matching. It does not extend to this project's **cross-sector, heterogeneous network** (roads, hospitals, substations, habitations, each with distinct flow types and failure semantics) at **rural-**

village scale — no dataset validates that specific structure, because it has rarely been built or observed at that scale in that setting.

- **Verified** means every accepted output is internally consistent — it obeys conservation laws, respects capacity bounds, and (where checked) matches the deterministic symbolic engine's exact computation.
- **Domain-adjacent validated** means the underlying mechanism has real-world empirical support in a related, better-instrumented domain (power grids).
- **Not validated for this project's specific extension** means no accepted output has been confirmed against a real, observed cascading *infrastructure* failure in comparable rural Indian, cross-sector terrain, because no such dataset exists yet.

This is not a flaw unique to this project — it is the standard situation for any model extending an established mechanism into a new, harder regime, and professional practice in this space (contingency analysis in power grids being the direct real-world analog this project's own core model is borrowed from) does not wait for a real disaster before a model becomes useful. Stating this distinction explicitly and precisely, in the pitch, before a judge extracts it, is the intended posture — and citing the 2012 India blackout as evidence the *mechanism* is real-world grounded is a stronger opening than claiming no grounding exists at all.

Additional stated limitations, consolidated:

- Dependency edges (Stage 3) are rule-inferred, not measured.
- Fragility thresholds (Stage 5) are grounded in engineering standards for roads, but rest on simplifying assumptions for some other asset classes.
- Load/capacity values (Stage 6) are demand-derived, not measured against real traffic counts for this specific district.
- Monte Carlo trial volume (Stage 7d) increases statistical stability of the model's own predictions; it does not increase real-world accuracy.
- The neural surrogate (Stage 9) can only approximate the symbolic engine it was trained on, never exceed its correctness.

11. Prototype Validation Benchmark Strategy

The cascading mechanism this engine implements (Mottet-Lai threshold-based overload redistribution) is not a novel, unvalidated invention — it is independently validated against real cascading failures in two adjacent, data-rich domains. Rather than claim this transfers automatically to the target rural, cross-sector domain, the prototype demonstrates the engine's soundness directly against available real-world-adjacent data, and explicitly names the remaining gap.

Evidence the mechanism is real-world grounded (cited, not run):

- The July 2012 India Northern/Eastern/North-Eastern grid collapse — the largest blackout in world history by population affected (~620-670 million people, 20+ states) — followed the textbook overload-redistribution "spiral breakdown" sequence this model formalizes, and is documented in an official Government of India Enquiry Committee report.
- Published cascading-failure models for urban road networks have been validated against real cities, including a sub-network of **Xi'an** (using collected real traffic data), real network data from **London** and **Shenyang**, and Shanghai's **Yangpu District** (using real public maps and identified real commuting corridors), each reproducing a genuine, sudden phase-transition-like collapse pattern consistent with the model's predictions.

Benchmark demonstrations run in the prototype (not the rural target district):

Tier	Dataset	What it demonstrates
A	A real, data-rich Indian city's road network (via <code>osmnx</code>), not the sparse rural target district)	The engine reproduces the qualitative signatures documented in validated road-cascade literature: sudden discontinuous collapse past a threshold, high sensitivity to a small number of bottleneck nodes, cascade-size distribution shape consistent with published studies
B	Standard IEEE 39-bus / 118-bus power test systems (via <code>pandapower</code> or <code>PyPSA</code>)	The domain-agnostic Tier 0/1 core, applied to a standard benchmark network used throughout the Motter-Lai literature, reproduces published cascading behavior (blackout-size distribution shape, critical-node rankings)
C	July 2012 India blackout (cited, not replicated — real transmission topology at that granularity is not public)	Narrative grounding: the mechanism matches a real, officially investigated Indian cascading infrastructure failure

What this does and does not establish: Tiers A and B confirm the engine's core mechanism behaves as expected on real or realistic, data-rich networks — a genuine, checkable claim. Neither tier validates the engine's specific application to a heterogeneous, cross-sector, rural-scale network, because no comparable real dataset exists for that combination. This is stated as the project's explicit open contribution, not implied to be already closed by the benchmark demonstrations above.

12. VARUNA Component Migration Reference

VARUNA component (original)	New system equivalent	Nature of change
Raster DTM graph	Heterogeneous OSM-based network graph (Stage 2)	Full replacement — different topology class
D8 grid adjacency	Vector-computed spatial + functional edges (Stage 2)	Full replacement — enables non-local edges
DUALFloodGNN	Cascade engine (Stage 7) + optional GAT surrogate (Stage 9); VARUNA itself becomes Stage 4	Repurposed as upstream input, new engine built on top
Continuity loss	Flow-conservation loss (Stage 9, if built)	Direct analog
Gravity alignment loss	Capacity-violation loss (Stage 9, if built)	Direct analog
Ponding threshold (0.3m)	Fragility curves per asset type (Stage 5)	Direct analog, generalized and made probabilistic
is_protected ecological flag	Empirical/centrality criticality score (Stage 8)	Same mechanism, different purpose
Dijkstra + ecological cost	Dijkstra for reachability + rerouting (Stage 7a)	Retained, repurposed
Digital Twin scenario simulation	Multi-scenario comparison engine (Stage 10)	Same concept, new metrics
Village demo CLI	District demo CLI	Same pattern, new scope

Document reflects the full architectural design process for this project's pivot from a single-domain hydrological digital twin to a cross-sector infrastructure cascading-failure resilience platform, built for MIT Tech Tattva, Problem Statement 11.